SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

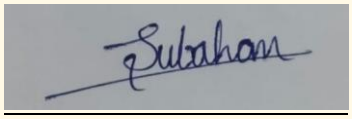
CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	T. Subahan	**Reg. No.:**	192511121
Date:	28/10/26	**Duration:**	60 minutes

Code of Conduct

I T. Subahan(192511121) *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student



Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. **Problem Analysis**
   - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
2. **Project Structure Design**
   - Design and explain the folder structure of your project repository.
   - Justify the purpose of each major folder and file included in the repository.
3. **Git Repository Initialization**
   - Create a Git repository for the project.
   - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
4. **Git Branching Strategy**
   - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
   - Justify why the selected branching model is appropriate for the assigned project.
5. **Version Control Workflow**
   - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.

- Explain the purpose of each Git operation performed.

6. **Commit History Analysis**
    - Present the commit history of the project.
    - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
    - Explain why Docker is suitable for the assigned project.
    - Identify the application components that require containerization.
8. **Dockerfile Development**
    - Design and explain the Dockerfile required to containerize the application.
    - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
    - Develop a Docker Compose configuration for the project.
    - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
    - Demonstrate the execution of the application using Docker containers.
    - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
    - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
    - Identify the challenges encountered during implementation.
    - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository &	Repository initialized	Repository and	Basic Git setup	Incorrect or incomplete Git

Branching Strategy(20%)	correctly with appropriate branching strategy, clear workflow, and proper repository organization.	branching strategy are mostly correct.	with limited branching implementation.	repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	**Mark**
Problem Analysis & Project Design (20%)	/5
Git Repository & Branching Strategy(20%)	/4
Version Control Workflow & Commit History(10%)	/4
Docker Implementation(20%)	/4
Deployment, Testing & Results(15%)	/4
Documentation, Challenges & Future Enhancements(15%)	/4
TOTAL	/25

Vehicle Service and Maintenance Management System Using Git and Docker

1. Problem Analysis

1.1 Problem Statement

The Vehicle Service and Maintenance Management System is a web-based application designed to help vehicle owners manage their vehicle servicing activities efficiently.

In the traditional approach, vehicle owners contact service centers through phone calls or visit them directly to book appointments. They may also find it difficult to maintain service records, remember upcoming maintenance dates, and track the progress of ongoing repairs.

The proposed system provides a centralized platform where vehicle owners can register their vehicles, book service appointments, view previous service records, receive maintenance reminders, and track the status of their vehicles.

Git is used to manage source code and support collaborative development, while Docker is used to package and run the application in a consistent environment.

1.2 Project Objectives

1. To develop a user-friendly vehicle service management application.
2. To allow vehicle owners to book service appointments online.
3. To maintain vehicle details and service history.
4. To provide maintenance reminders based on service dates or vehicle usage.
5. To enable users to track service progress.
6. To use Git for version control and collaborative development.
7. To use Docker to containerize the application and simplify deployment.

1.3 Functional Requirements

S. No.	Functional Requirement	Description
1	Vehicle Registration	Add and manage vehicle details.
2	Appointment Booking	Book a vehicle servicing appointment.
3	Service History	View previous service records.
4	Maintenance Reminders	Display upcoming maintenance requirements.
5	Service Status Tracking	Check whether a vehicle is booked, under service, or ready for collection.
6	Appointment Management	View or cancel appointments.
7	User Management	Register and log in to the application.

1.4 Non-Functional Requirements

- **Performance:** The application should respond to user requests quickly.
- **Security:** User credentials and vehicle information must be protected.

- **Reliability:** The application should operate consistently and handle errors appropriately.
- **Usability:** The interface should be simple and easy to understand.
- **Maintainability:** The code should be organized into manageable modules.
- **Portability:** The application should run in different environments using Docker.
- **Scalability:** The architecture should support additional users and features as the project grows.

1.5 Expected Outcomes

The project is expected to provide a centralized platform for managing vehicle servicing activities. It should reduce manual recordkeeping, simplify appointment booking, improve service tracking, and help vehicle owners remember maintenance requirements.

The project also demonstrates how Git and Docker can be integrated into the software development lifecycle.

2. Project Structure Design

2.1 Proposed Repository Structure

The project uses a simple full-stack architecture with a frontend, backend, and database.

```
vehicle-service-management/
│
├── frontend/
│   ├── index.html
│   ├── style.css
│   └── script.js
│
├── backend/
│   ├── server.js
│   ├── package.json
│   └── package-lock.json
│
├── database/
│   └── init.sql
│
├── screenshots/
│   ├── git-branches.png
│   ├── git-commit-history.png
│   └── docker-containers.png
│
├── Dockerfile
├── docker-compose.yml
├── .dockerignore
├── .gitignore
```

```
├── .env.example
└── README.md
```

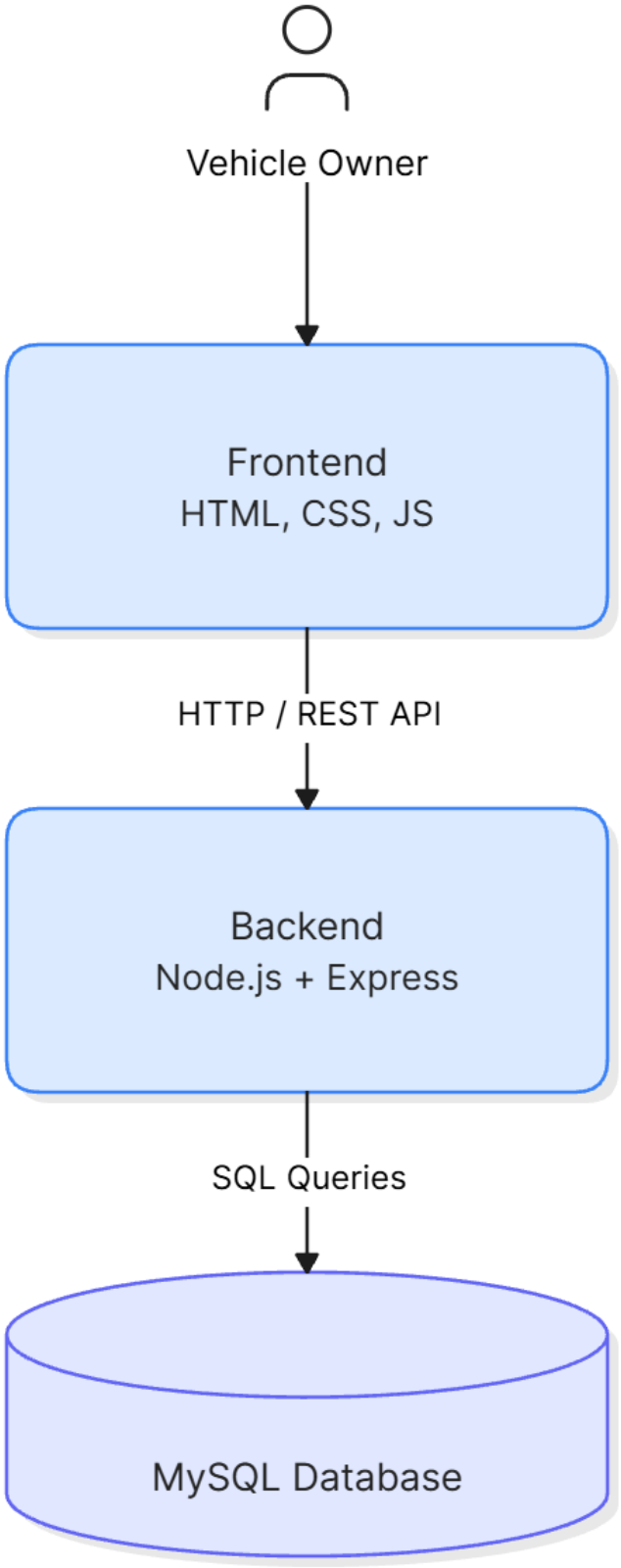
Note: This is the proposed repository structure. Create the folders and files as you implement the application.

2.2 Purpose of Each Folder and File

Folder/File	Purpose
frontend/	Contains the user interface files.
index.html	Defines the application's webpage structure.
style.css	Controls the appearance and layout of the application.
script.js	Handles frontend interactions and communicates with the backend.
backend/	Contains the server-side application code.
server.js	Implements API endpoints and application logic.
package.json	Defines backend dependencies and execution scripts.
package-lock.json	Records the resolved Node.js dependency versions.
database/	Stores database initialization scripts.
init.sql	Creates the required database tables.
screenshots/	Stores evidence of Git operations and Docker execution.
Dockerfile	Defines how the application image is built.
docker-compose.yml	Defines and coordinates the application services.
.dockerignore	Excludes unnecessary files from the Docker build context.
.gitignore	Prevents temporary files and secrets from being committed to Git.
.env.example	Documents the environment variables required to run the project.
README.md	Provides project information and setup instructions.

2.3 Architecture Diagram

The system follows a three-layer architecture.



Explanation

1. **Frontend:** Allows vehicle owners to interact with the application.
2. **Backend:** Processes requests, validates input, and performs service-management operations.
3. **Database:** Stores vehicle details, appointments, and service history.

Docker Compose connects the backend and database through an internal network. The frontend can be served by the backend in this implementation.

3. Git Repository Initialization

3.1 Introduction

Git is a distributed version control system that tracks changes in source code. It allows developers to maintain different versions of a project, collaborate with team members, and restore earlier versions when necessary.

A remote repository on GitHub can be used to store the project and synchronize changes between developers.

3.2 Steps to Initialize the Repository

Open a terminal or Git Bash and execute the following commands.

Step 1: Create the project folder

mkdir vehicle-service-management
cd vehicle-service-management

Step 2: Initialize Git

git init

This command creates a local Git repository.

Step 3: Configure the Git username and email

git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"

Replace these sample values with your own Git identity.

Step 4: Create the project folders

mkdir frontend backend database screenshots

Create the project files according to the proposed folder structure.

Step 5: Check the repository status

git status

This command displays untracked files and modified files.

Step 6: Stage the project files

git add .

Staging selects the changes that will be included in the next commit. Before running this command, ensure that credentials, .env files, and other sensitive information are excluded.

Step 7: Create the first commit

git commit -m "Initial project structure"

A commit saves a snapshot of the staged changes in the local repository.

3.3 Essential Git Files and Concepts

- **.git/:** A hidden directory created by Git to store repository metadata, object data, references, and configuration.
- **.gitignore:** Specifies untracked files that Git should ignore.
- **README.md:** Describes the project, setup steps, and usage instructions. It is a project documentation file, not a special Git file.
- **Commit:** A recorded snapshot of the project's staged changes.
- **Branch:** An independent line of development.
- **Remote:** A reference to a repository hosted on a service such as GitHub.

Example .gitignore file:

```
node_modules/
.env
.DS_Store
coverage/
npm-debug.log*
```

This prevents dependency folders, local secrets, and unnecessary generated files from being tracked.

4. Git Branching Strategy

4.1 Selected Strategy

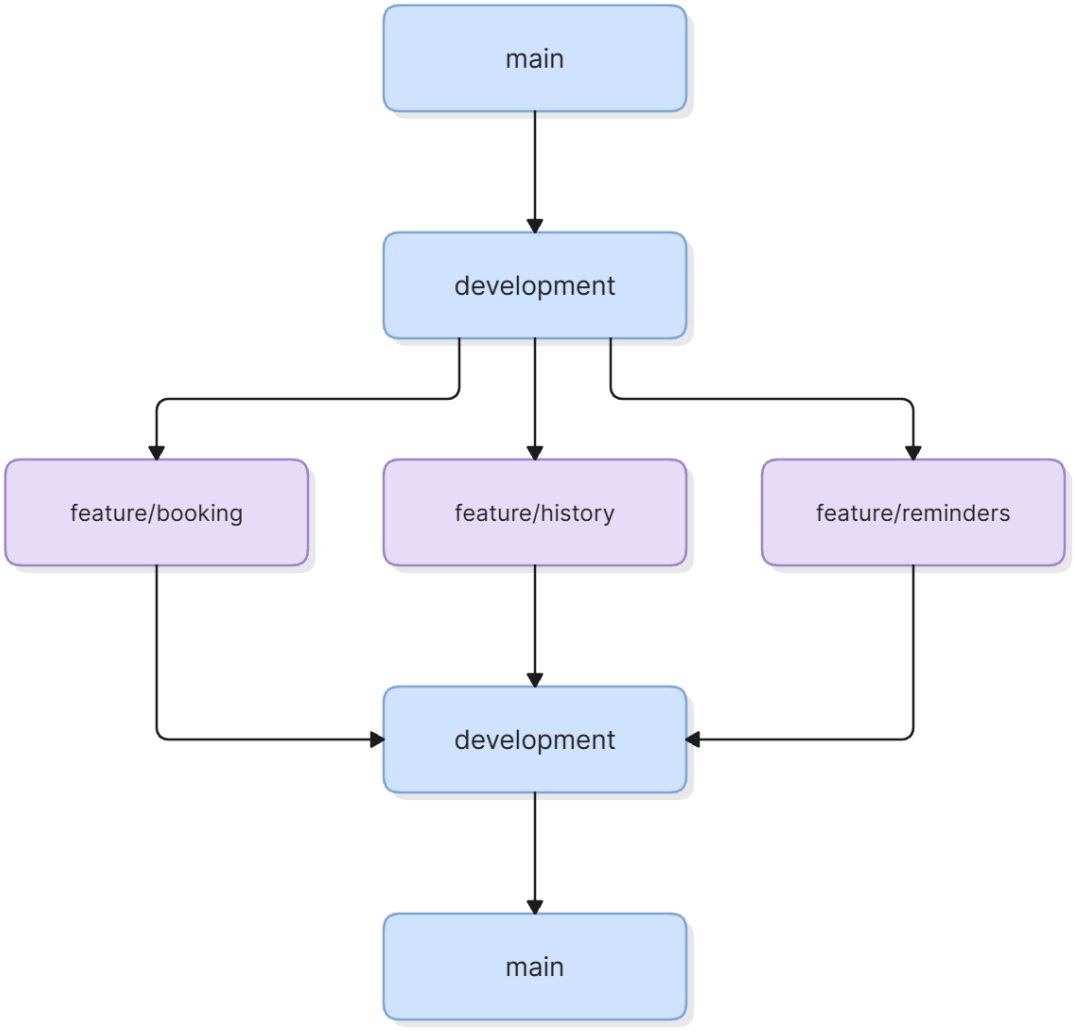
The project uses a simplified feature-branch workflow.

The main branches are:

- **Main:** Contains stable versions of the application.
- **Development:** Integrates completed features before they are merged into the main branch.
- **Feature branches:** Used to develop individual features independently.
- **Hotfix branches:** Used to correct urgent problems in a stable release.

A separate release branch may be created if the project requires a formal testing and release stage.

4.2 Branching Diagram



4.3 Creating Branches

Create the development branch:
git switch -c development
Create a feature branch for appointment booking:
git switch -c feature/appointment-booking
After implementing the feature, stage and commit the changes:
git add .
git commit -m "Add appointment booking feature"
Return to the development branch:
git switch development
Merge the feature branch:
git merge --no-ff feature/appointment-booking
After testing the integrated application, merge the development branch into main:
git switch main

git merge --no-ff development

4.4 Justification

This strategy is appropriate because the project contains multiple independent features, such as appointment booking, vehicle registration, service history, and maintenance reminders.

Feature branches reduce interference between changes. The development branch provides a place to integrate and test features before release. The main branch remains the stable version of the project.

For a small academic project, this simplified strategy provides the benefits of organized development without introducing unnecessary workflow complexity.

5. Version Control Workflow

5.1 Git Workflow

The Git workflow consists of the following steps:

1. Create or modify source code.
2. Check the working tree using git status.
3. Stage the required changes using git add.
4. Record the changes using git commit.
5. Push the branch to the remote repository.
6. Review and merge changes.
7. Synchronize the local repository with the remote repository.

5.2 Important Git Commands

Command	Purpose
git init	Initializes a local repository.
git status	Displays the working-tree and staging status.
git add .	Stages eligible changes.
git commit -m "message"	Records staged changes.
git branch	Lists local branches.
git switch branch-name	Switches to an existing branch.
git switch -c branch-name	Creates and switches to a new branch.
git merge branch-name	Merges another branch into the current branch.
git log --oneline --graph --all	Displays a compact commit graph.
git remote add origin URL	Adds a remote repository.
git push -u origin main	Pushes main and sets its upstream.
git pull --rebase	Fetches and rebases local work on top of the upstream branch.

5.3 Connecting to GitHub

Create an empty repository on GitHub named vehicle-service-management.

Then execute:

git remote add origin https://github.com/YOUR_USERNAME/vehicle-service-management.git

git branch -M main
git push -u origin main
Replace YOUR_USERNAME with your GitHub username.
After creating the development branch, publish it:
git switch development
git push -u origin development
For subsequent updates:
git add .
git commit -m "Add service history module"
git push

5.4 Demonstrating Conflict Resolution

A merge conflict can occur when two branches modify the same part of a file differently.
For example, one developer changes the service status to In Progress, while another changes it to Under Repair.
Git may mark the conflicting section as follows:

```
<<<<<<< HEAD
status = "In Progress";
=======
status = "Under Repair";
>>>>>>> feature/service-status
```

To resolve the conflict:

1. Open the affected file.
2. Decide which change is appropriate, or combine both changes.
3. Remove the conflict markers.
4. Test the corrected code.
5. Stage and commit the resolution.

git add backend/server.js
git commit -m "Resolve service status merge conflict"
The example demonstrates the conflict-resolution process. Only report that a conflict occurred in your project if you actually reproduce and resolve one.

6. Commit History Analysis

6.1 Importance of Commit History

Commit history records how the project changes over time. It helps developers identify when features were introduced, investigate bugs, review previous implementations, and understand contributions.

6.2 Recommended Commit Messages

The following is an example of a planned commit sequence. These entries are illustrative; your actual report should contain the history generated by your repository.
feat: add vehicle registration form

feat: implement appointment booking
feat: add service history module
feat: implement service status tracking
fix: validate appointment date
docs: update project setup instructions
build: add Dockerfile and Compose configuration
test: verify backend health endpoint

6.3 Good Commit Message Practices

1. Use short, meaningful descriptions.
2. Begin with an action or a consistent prefix.
3. Keep each commit focused on a logical change.
4. Avoid vague messages such as changes, update, or final.
5. Separate unrelated features and bug fixes into different commits.
6. Do not commit passwords, API keys, or other secrets.

For example, feat: add appointment booking is more useful than done.

6.4 Displaying the Commit History

Execute:

git log --oneline

For a graphical view of branches and merges:

git log --oneline --graph --decorate --all

Screenshot to include: Capture the terminal output showing your actual commit messages, branches, and merge history.

7. Docker Environment Design

7.1 Introduction to Docker

Docker is a containerization platform that packages an application and its dependencies into containers. Containers provide a consistent runtime environment, reducing differences between development and deployment systems.

In this project, Docker is used to run the backend and database in separate containers.

7.2 Why Docker Is Suitable for This Project

The Vehicle Service and Maintenance Management System requires an application server and a database. Installing and configuring these components separately on every computer can be time-consuming.

Docker simplifies the process by providing predefined configurations for the application environment.

The main benefits are:

- **Portability:** The application can run on different systems that support Docker.
- **Consistency:** Developers can use the same configured environment.
- **Isolation:** The backend and database run in separate containers.
- **Easy deployment:** Services can be started with a single Compose command.

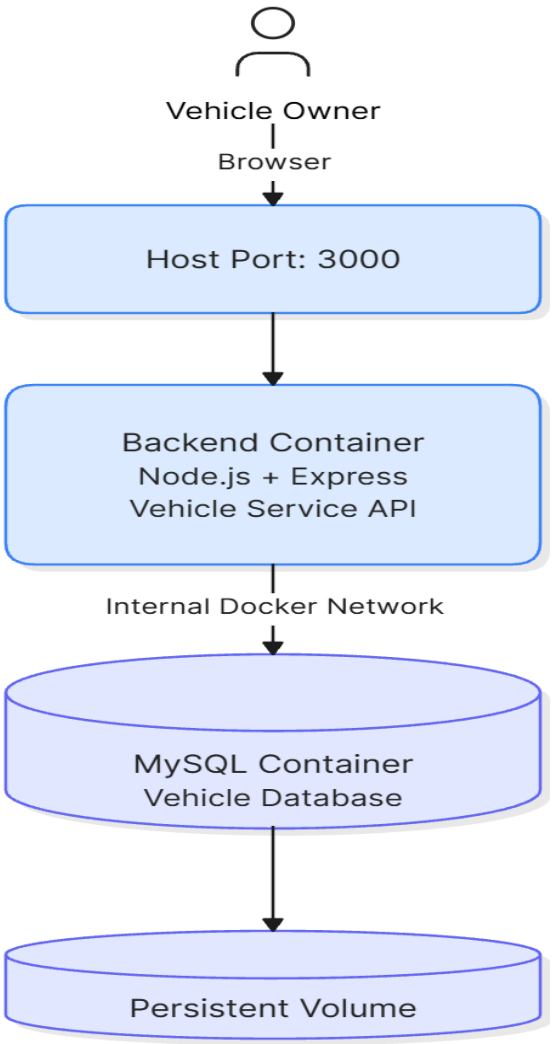
- **Maintainability:** Application components can be updated independently.
- **Scalability:** The architecture can be extended to run additional service instances when required.

7.3 Components Requiring Containerization

Component	Technology	Purpose
Backend	Node.js and Express	Processes application requests and implements business logic.
Database	MySQL	Stores vehicle information, appointments, and service history.
Frontend	HTML, CSS, JavaScript	Provides the user interface.

For this implementation, the frontend is served by the backend, so a separate frontend container is not necessary.

7.4 Docker Architecture Diagram



The backend is accessible through the host's port 3000. The database uses an internal Docker network, so the application can communicate with it without exposing the MySQL port to the host.

8. Dockerfile Development

8.1 Purpose of a Dockerfile

A Dockerfile contains instructions for building a Docker image. The image provides the runtime environment and application files required to start a container.

For this project, the Dockerfile defines the environment for the Node.js backend.

8.2 Sample Backend Dockerfile

Create a file named Dockerfile in the root project directory.

```
FROM node:22-alpine

WORKDIR /app

COPY backend/package*.json ./

RUN npm ci

COPY backend/ ./

EXPOSE 3000

CMD ["node", "server.js"]
```

Important: This example assumes that backend/package-lock.json exists and matches backend/package.json. Generate and commit the lockfile before building the image.

8.3 Explanation of Dockerfile Instructions

Instruction	Explanation
FROM node:22-alpine	Uses a lightweight Node.js 22 Alpine base image.
WORKDIR /app	Sets /app as the working directory inside the container.
COPY backend/package*.json ./	Copies the backend dependency manifests into the image.
RUN npm ci	Installs the dependencies using the lockfile.
COPY backend/ ./	Copies the backend source code into the image.
EXPOSE 3000	Documents that the application listens on port 3000.
CMD ["node", "server.js"]	Starts the Node.js application when the container launches.

Why Use a Lightweight Base Image?

The Alpine-based image is relatively small, which can reduce image storage requirements and download time. The exact size depends on the Node.js version, architecture, and installed

dependencies.

8.4 Sample .dockerignore File

Create .dockerignore in the project root.

```
.git
.gitignore
node_modules
**/node_modules
.env
screenshots
npm-debug.log*
```

This file prevents unnecessary files and local secrets from being included in the Docker build context.

Security note: Do not copy production passwords or secret keys into the image. Supply sensitive configuration at runtime through an appropriate secrets-management mechanism.

9. Docker Compose Design

9.1 Introduction

Docker Compose is a tool for defining and managing applications containing multiple containers.

The project uses Docker Compose to coordinate the Node.js backend and MySQL database. It defines the services, network, persistent database storage, environment variables, and startup dependencies.

9.2 Docker Compose Configuration

Create a file named docker-compose.yml in the project root.

```
services:
  backend:
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - "3000:3000"
    environment:
      PORT: "3000"
      DB_HOST: db
      DB_PORT: "3306"
      DB_NAME: vehicle_service
      DB_USER: vehicle_user
      DB_PASSWORD: ${DB_PASSWORD:?Set DB_PASSWORD in your environment}
    depends_on:
      db:
```

```
      condition: service_healthy
    restart: unless-stopped
    networks:
      - vehicle-network

  db:
    image: mysql:8.4
    environment:
      MYSQL_DATABASE: vehicle_service
      MYSQL_USER: vehicle_user
      MYSQL_PASSWORD: ${DB_PASSWORD:?Set DB_PASSWORD in your environment}
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD:?Set DB_ROOT_PASSWORD in your environment}
    volumes:
      - mysql-data:/var/lib/mysql
    healthcheck:
      test:
        - CMD-SHELL
        - mysqladmin ping -h localhost -u root -p"$${MYSQL_ROOT_PASSWORD}" --silent
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 30s
    restart: unless-stopped
    networks:
      - vehicle-network

networks:
  vehicle-network:

volumes:
  mysql-data:
```

This configuration requires the application to read the database settings from environment variables and connect using the hostname db.

The backend must also implement appropriate database connection handling and wait for the database to become available.

9.3 Environment Variables

Create a local .env file in the project root.

```
DB_PASSWORD=ChangeThisToAStrongPassword
```

DB_ROOT_PASSWORD=ChangeThisToAnotherStrongPassword
Replace the example values with strong, unique passwords.
Do not commit .env to Git. The earlier .gitignore configuration excludes it.
Create .env.example containing placeholder values for documentation:
DB_PASSWORD=replace_with_database_password
DB_ROOT_PASSWORD=replace_with_root_password
The example file can be committed because it contains placeholders rather than real credentials.

9.4 Explanation of Compose Components

Component	Purpose
backend	Builds and runs the Node.js application.
db	Runs the MySQL database server.
build	Builds the backend image using the Dockerfile.
ports	Maps host port 3000 to container port 3000.
environment	Supplies application and database configuration.
depends_on	Delays backend startup until the database passes its health check.
healthcheck	Checks whether MySQL is responding.
networks	Allows the backend to communicate with the database.
volumes	Preserves MySQL data beyond the lifetime of a container.
restart	Restarts containers automatically under the specified conditions.

Database Initialization

The project can use database/init.sql to create additional tables.
To execute it automatically when the MySQL database is initialized for the first time, add this section to the db service:

```
volumes:
  - mysql-data:/var/lib/mysql
  - ./database/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
```

This is an additional configuration for the same db service, not a separate service definition.
MySQL initialization scripts run when the database is initialized with an empty data directory. Adding or modifying the script later does not automatically rerun it against an existing database volume.

10. Container Deployment and Testing

10.1 Prerequisites

Before running the application, install:

1. Git.
2. Docker Desktop on Windows, or Docker Engine with the Compose plugin on a supported Linux system.
3. A code editor such as Visual Studio Code.

Verify the installations:

git --version

docker --version

docker compose version

10.2 Build the Docker Image

From the project root, execute:

docker compose build

This command builds the backend image using the Dockerfile.

10.3 Start the Containers

Execute:

docker compose up -d

This starts the backend and database containers in the background.

To inspect the running containers:

docker compose ps

To view application logs:

docker compose logs -f backend

To view database logs:

docker compose logs -f db

Press Ctrl+C to stop following the logs. This does not normally stop the containers.

10.4 Test the Application

The following tests should be performed after implementing the corresponding backend functionality.

Test Case	Procedure	Expected Result
TC01	Start the application using Docker Compose.	Backend and database start successfully.
TC02	Open http://localhost:3000 in a browser.	The application page loads if the frontend is configured to be served at /.
TC03	Register a vehicle.	Valid vehicle details are accepted and saved.
TC04	Book a service appointment.	The appointment is created and stored in the database.
TC05	View service history.	Previously stored service records are displayed.
TC06	Update service status.	The latest status is displayed to the user.
TC07	Restart the containers.	Previously stored database records remain available.
TC08	Submit invalid appointment details.	The application rejects invalid data with an appropriate message.

Note: The expected results above are test objectives, not claims that the tests have already passed. Record the actual results after running the application.

10.5 Useful Testing Commands

Check the running containers:

```
docker compose ps
```

Check the backend logs:

```
docker compose logs backend
```

Check the database logs:

```
docker compose logs db
```

Check the application's HTTP response if a root route is implemented:

```
curl -i http://localhost:3000
```

If a health endpoint such as /health is implemented, test it with:

```
curl -i http://localhost:3000/health
```

The health endpoint must be implemented in the backend before this test can succeed.

To stop the application:

```
docker compose down
```

This stops and removes the containers and network created by Compose. The named database volume remains.

To remove the database volume as well, use:

```
docker compose down -v
```

Warning: The second command deletes the persistent database volume and its stored data. Do not use it when you need to preserve your records.

10.6 Screenshots to Include in the Report

Capture your own screenshots of the following operations:

1. Git repository initialization using git init.
2. Project branch creation using git branch or git branch --all.
3. Successful commits using git log --oneline --graph --all.
4. GitHub repository showing the project files.
5. Docker image build output.
6. Running containers using docker compose ps.
7. Backend logs showing successful startup.
8. The application running in a browser.
9. A successful appointment booking or service history operation.
10. Database persistence after restarting the containers.

Store the screenshots in the screenshots/ directory and reference them in the corresponding report sections.

11. Benefits of Git and Docker

Git and Docker support different but complementary parts of the software development process.

11.1 Benefits of Git

Version management: Git tracks source code changes and maintains a history of commits.
Collaboration: Multiple developers can work on separate branches and combine their work.
Conflict resolution: Git identifies conflicting changes and provides tools to resolve them.
Accountability: Commit history helps identify which changes were made and when.
Recovery: Previous commits can be inspected or restored when necessary.
Code maintenance: Meaningful commits and organized branches make the project easier to understand.

11.2 Benefits of Docker

Portability: The application can run consistently on different supported environments.
Environment consistency: Developers can use the same application runtime and database versions.
Isolation: The backend and database run in separate containers.
Simplified deployment: Docker Compose starts the required services using a single command.
Persistent storage: Docker volumes preserve database files when containers are recreated.
Extensibility: Additional components, such as a notification service or reverse proxy, can be added when needed.

11.3 Combined Benefits

Area	Contribution of Git	Contribution of Docker
Collaboration	Supports branching and merging.	Provides a consistent environment for developers.
Version management	Records source code changes.	Allows application images and dependencies to be versioned.
Portability	Makes source code available across development systems.	Packages runtime dependencies into images.
Deployment	Maintains deployment configuration in version control.	Runs the application and database as containers.
Scalability	Helps manage changes to the architecture.	Supports running additional service instances when designed appropriately.
Maintainability	Provides traceable changes and commit history.	Separates application components and their environments.

Therefore, Git improves source code management, while Docker simplifies application execution and deployment. Together, they improve the reliability and maintainability of the Vehicle Service and Maintenance Management System.

12. Challenges and Improvements

12.1 Challenges Encountered or Anticipated

The following are common challenges that may arise while implementing this assignment. Include only the challenges you actually encountered in your practical work as completed observations.

1. Git merge conflicts

Different developers may modify the same section of a file, resulting in a merge conflict.

Solution: Use feature branches, commit changes frequently, and review conflicting sections before merging.

2. Docker image build failures

The image build may fail if the Dockerfile references missing files, the dependency lockfile is unavailable, or dependencies cannot be installed.

Solution: Verify file paths, maintain the lockfile, and inspect the Docker build output.

3. Database connectivity problems

The backend may attempt to connect before MySQL is ready or may use an incorrect hostname.

Solution: Use the Compose service name db, configure a database health check, and implement connection retry handling.

4. Environment configuration errors

Incorrect database credentials or missing environment variables may prevent the application from starting.

Solution: Document required variables in .env.example and validate the configuration at startup.

5. Data loss during container recreation

Database records may be lost if persistent storage is not configured correctly.

Solution: Use a named Docker volume and maintain appropriate database backups.

6. Different development environments

Developers may use different runtime versions or dependency configurations.

Solution: Use a Dockerfile, a dependency lockfile, and documented setup instructions.

12.2 Future Improvements

The project can be improved in the following ways:

1. **Automated testing:** Add unit tests and integration tests for appointment booking, vehicle registration, and service tracking.
2. **Continuous integration:** Configure GitHub Actions to run tests and validate Docker builds whenever code is pushed.
3. **Security improvements:** Implement secure authentication, authorization, input validation, and safe secret management.
4. **Automated deployment:** Deploy tested application images to a suitable hosting environment.
5. **Notification service:** Add email or SMS notifications for upcoming maintenance and appointment updates.
6. **Database backup:** Implement scheduled backups and a recovery procedure.
7. **Monitoring:** Add application health checks, logging, and performance monitoring.
8. **Scalability:** Introduce a reverse proxy, load balancing, and additional backend instances if demand increases.
9. **Code review:** Require pull requests and reviews before merging feature branches into the

main branch.

10. **Container optimization:** Use a non-root application user, minimize unnecessary image contents, and scan images for vulnerabilities.

Conclusion

The Vehicle Service and Maintenance Management System provides a practical example of how version control and containerization can be applied to a full-stack application.

Git is used to manage source code, maintain commit history, create feature branches, resolve conflicts, and synchronize changes with a remote repository. Docker packages the backend and its runtime environment, while Docker Compose coordinates the backend and MySQL database.

The combined use of Git and Docker improves collaboration, consistency, portability, deployment, and maintainability. Future enhancements such as automated testing, continuous integration, notifications, and monitoring can further improve the project.